Homework 08

The former kernel-tuning question has moved to Homework 09.

Solutions for Questions 1 and 2 will be available after Monday, October 19, 2026. No solution can be provided for Question 3.

For Questions 1 and 2, use either R or Python.

Question 1: Boundary and margin

Use `margin-geometry-20.csv` from the combined ZIP, which contains 20 observations with predictors x_1, x_2 and class labels $y \in \{-1, 1\}$. Consider the score

$$f(x) = 0.8x_1 + 0.6x_2.$$

Since $\|(0.8, 0.6)\|_2 = 1$, the lines $f(x) = \pm 1$ are each a distance 1 from the decision boundary $f(x) = 0$.

a. Calculate the distance from every observation to the decision boundary and its functional margin $yf(x)$. In a table, report these values and whether each observation is inside, on, or beyond the margin for its class. Compare $yf(x)$ with 1 to make this distinction.

```
geometry <- read.csv("data/margin-geometry-20.csv")

geometry$score <- 0.8 * geometry$x1 + 0.6 * geometry$x2

# Perpendicular distance to the boundary
geometry$distance <- abs(geometry$score) / sqrt(0.8^2 + 0.6^2)

# Functional margin
geometry$functional_margin <- geometry$y * geometry$score

# Use a tolerance for floating-point equality
tol <- 1e-8

geometry$position <- ifelse(
  abs(geometry$functional_margin - 1) < tol,
  "On",
  ifelse(geometry$functional_margin < 1, "Inside", "Beyond")
)

results <- geometry[, c(
  "observation", "distance", "functional_margin", "position"
)]

knitr::kable(results, digits = 2)
```

```
Warning: 'xfun::attr()' is deprecated.
Use 'xfun::attr2()' instead.
See help("Deprecated")

Warning: 'xfun::attr()' is deprecated.
Use 'xfun::attr2()' instead.
See help("Deprecated")
```

observation	distance	functional_margin	position
1	1.00	1.00	On
2	1.35	1.35	Beyond
3	1.50	1.50	Beyond
4	1.70	1.70	Beyond
5	1.90	1.90	Beyond
6	2.20	2.20	Beyond
7	2.40	2.40	Beyond
8	0.64	0.64	Inside
9	0.20	0.20	Inside
10	0.20	-0.20	Inside
11	1.00	1.00	On
12	1.30	1.30	Beyond
13	1.55	1.55	Beyond
14	1.75	1.75	Beyond
15	1.95	1.95	Beyond
16	2.15	2.15	Beyond
17	2.45	2.45	Beyond
18	0.64	0.64	Inside
19	0.24	0.24	Inside
20	0.20	-0.20	Inside

The perpendicular distance from an observation to the decision boundary is

$$d(x) = \frac{|0.8x_1 + 0.6x_2|}{\sqrt{0.8^2 + 0.6^2}} = |f(x)|.$$

Its functional margin is $yf(x)$. An observation is inside its class margin when $yf(x) < 1$, on its class margin when $yf(x) = 1$, and beyond its class margin when $yf(x) > 1$. Negative functional margins indicate misclassification.

b. Plot the observations, the boundary $f(x) = 0$, and the margin lines $f(x) = \pm 1$. Distinguish the two classes and identify the observations inside the margin in your plot.

```
class_color <- ifelse(geometry$y == 1, "steelblue", "firebrick")
class_symbol <- ifelse(geometry$y == 1, 16, 17)

plot(
  geometry$x1, geometry$x2,
  type = "n",
  xlim = c(-3.2, 3.2),
  ylim = c(-3, 3),
  xaxs = "i",
  yaxs = "i",
  asp = 1,
  xlab = expression(x[1]),
  ylab = expression(x[2]),
  main = "Decision boundary and class margins"
)

# f(x) = 0: x2 = -(0.8/0.6)x1
abline(a = 0, b = -0.8 / 0.6, lwd = 2)

# f(x) = +1 and f(x) = -1
abline(a = 1 / 0.6, b = -0.8 / 0.6, lty = 2, lwd = 2)
abline(a = -1 / 0.6, b = -0.8 / 0.6, lty = 2, lwd = 2)

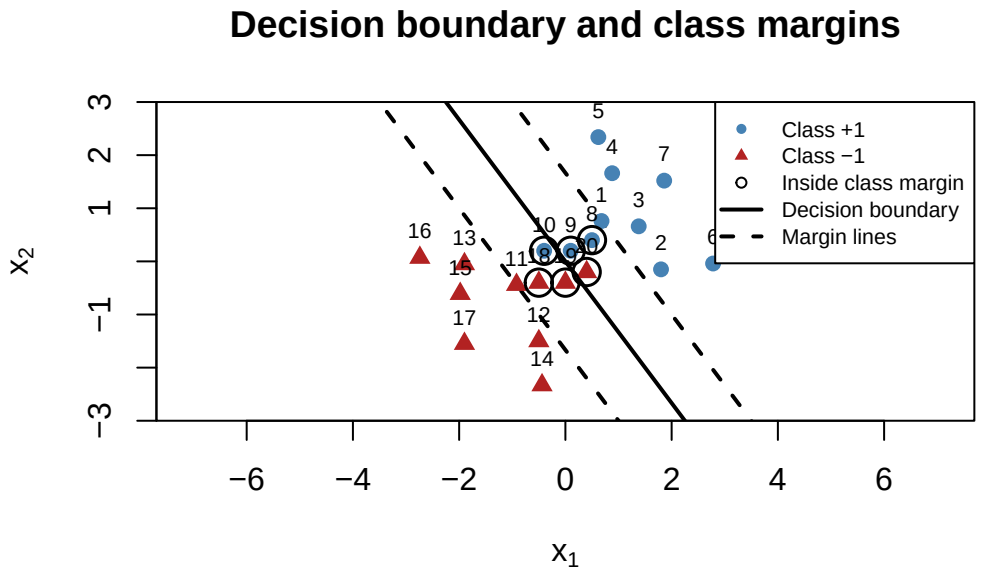
points(
  geometry$x1, geometry$x2,
  col = class_color,
  pch = class_symbol,
  cex = 1.1
)

# Circle all observations inside their class margin
inside <- geometry$position == "Inside"
```

```r
points(
  geometry$x1[inside],
  geometry$x2[inside],
  pch = 1,
  cex = 1.9,
  lwd = 1.5
)

# Label points using the supplied observation numbers
text(
  geometry$x1, geometry$x2,
  labels = geometry$observation,
  pos = 3,
  cex = 0.7,
  offset = 0.5
)

legend(
  "topright",
  legend = c(
    "Class +1", "Class -1", "Inside class margin",
    "Decision boundary", "Margin lines"
  ),
  col = c("steelblue", "firebrick", "black", "black", "black"),
  pch = c(16, 17, 1, NA, NA),
  lty = c(NA, NA, NA, 1, 2),
  lwd = c(NA, NA, NA, 2, 2),
  cex = 0.7,
  bg = "white"
)
```



The solid line represents the decision boundary $f(x) = 0$, and the dashed lines represent $f(x) = 1$ and $f(x) = -1$. Blue circles indicate class $+1$, and red triangles indicate class -1. Black rings identify observations inside their class margin: 8, 9, 10, 18, 19, and 20. The plot uses equal units on both axes.

c. Report how many observations are inside, on, and beyond the margin. Which observations are misclassified by this decision rule?

```r
table(
  factor(geometry$position, levels = c("Inside", "On", "Beyond"))
)
```

```
Inside     On Beyond
     6      2     12
```

```
# A negative functional margin means misclassification
misclassified <- geometry$observation[
  geometry$functional_margin < 0
]

misclassified
```

```
[1] 10 20
```

There are **6 observations inside**, **2 on**, and **12 beyond** their class margins. Observations 1 and 11 are on the margin.

Observations **10 and 20 are misclassified** because their functional margins are negative. Observation 10 belongs to class $+1$ but has $f(x) = -0.20$, while observation 20 belongs to class -1 but has $f(x) = 0.20$. The misclassification rate is $2/20 = 10\%$.

Use the observation numbers supplied in the data. Set the x_1 axis to $[-3.2, 3.2]$, set the x_2 axis to $[-3, 3]$, and use an equal aspect ratio. Do not fit a new classifier for this question.

Question 2: Margin in four dimensions

Use `margin-geometry-4d.csv` from the combined ZIP, which contains 50 observations with four predictors and class labels $y \in \{-1, 1\}$. Consider the linear score

$$f(\mathbf{x}) = \beta_0 + \mathbf{x}^\mathsf{T}\beta, \qquad \beta_0 = -1, \quad \beta = (1, 2, -2, 4)^\mathsf{T}.$$

a. Project the observations onto the direction of β and plot their positions on one axis. Distinguish the two classes and mark the decision boundary. Use the plot to calculate the geometric margin M, the distance from the boundary to the closest observation.

```
geometry4 <- read.csv("data/margin-geometry-4d.csv")

X4 <- as.matrix(geometry4[, c("x1", "x2", "x3", "x4")])

beta <- c(1, 2, -2, 4)
beta0 <- -1
beta_norm <- sqrt(sum(beta^2))

projection <- drop(X4 %*% beta) / beta_norm

boundary <- -beta0 / beta_norm

score <- beta0 + drop(X4 %*% beta)
distance <- abs(score) / beta_norm

M <- min(distance)

closest <- geometry4$observation[
  abs(distance - M) < 1e-8
]

beta_norm
```

```
[1] 5
```

```
boundary
```

```
[1] 0.2
```

```
M
```

```
[1] 1
```

```
closest
```

```
[1]  4 15 33 36
```

```
colors <- ifelse(geometry4$y == 1, "steelblue", "firebrick")
symbols <- ifelse(geometry4$y == 1, 16, 17)

plot(
  projection, rep(0, nrow(geometry4)),
  col = colors,
  pch = symbols,
  xlim = range(c(projection, boundary)) + c(-0.3, 0.3),
  ylim = c(-0.3, 0.5),
  yaxt = "n",
  ylab = "",
  xlab = expression(t == x^T * beta / group("||", beta, "||")),
  main = "Projection onto the direction of beta"
)

abline(h = 0, col = "gray70")
abline(v = boundary, lwd = 2)
abline(v = boundary + c(-M, M), lty = 2)

# Ring and label the closest observations
is_closest <- geometry4$observation %in% closest

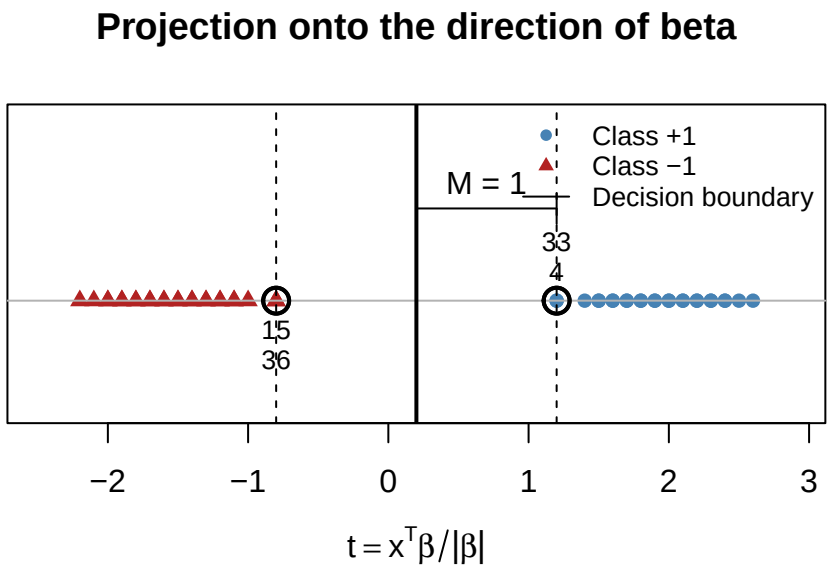
points(
  projection[is_closest],
  rep(0, sum(is_closest)),
  pch = 1,
  cex = 1.8,
  lwd = 2
)

text(
  projection[is_closest],
  c(0.08, -0.08, 0.16, -0.16),
  labels = geometry4$observation[is_closest],
  cex = 0.8
)

arrows(
  boundary, 0.25,
  boundary + M, 0.25,
  code = 3,
  angle = 90,
  length = 0.08
)

text(boundary + M / 2, 0.32, "M = 1")

legend(
  "topright",
  legend = c("Class +1", "Class -1", "Decision boundary"),
  col = c("steelblue", "firebrick", "black"),
  pch = c(16, 17, NA),
  lty = c(NA, NA, 1),
  bty = "n",
  cex = 0.8
)
```



The coefficient vector has length

$$\|\beta\| = \sqrt{1^2 + 2^2 + (-2)^2 + 4^2} = 5.$$

The projection coordinate along its unit direction is

$$t_i = \frac{x_i^\top \beta}{5}.$$

Since $f(x) = -1 + 5t$, the decision boundary is at $t = 0.2$. The closest observations are 4 and 33 at $t = 1.2$, and 15 and 36 at $t = -0.8$. Therefore, the geometric margin is

$$M = \min_i \frac{|f(x_i)|}{\|\beta\|} = \frac{5}{5} = \boxed{1}.$$

All observations are correctly classified by the supplied decision boundary. Points sharing a projection coordinate overlap in the plot.

b. Express the same decision boundary using the SVM normalization in which the smallest functional margin $y_i f(\mathbf{x}_i)$ is set to 1. What are the corresponding values of β and β_0?

```
functional_margin <- geometry4$y * score
smallest_margin <- min(functional_margin)

beta_svm <- beta / smallest_margin
beta0_svm <- beta0 / smallest_margin

smallest_margin
```

```
[1] 5
```

```
beta_svm
```

```
[1]  0.2  0.4 -0.4  0.8
```

```
beta0_svm
```

```
[1] -0.2
```

```r
normalized_score <- beta0_svm + drop(X4 %*% beta_svm)

min(geometry4$y * normalized_score)
```

```
[1] 1
```

```r
1 / sqrt(sum(beta_svm^2))
```

```
[1] 1
```

The smallest functional margin under the original coefficients is

$$\min_i y_i f(x_i) = 5.$$

To make this minimum equal to 1, divide both the intercept and coefficient vector by 5:

$$\boxed{\beta_0^* = -0.2}, \qquad \boxed{\beta^* = (0.2,\ 0.4,\ -0.4,\ 0.8)^\top}.$$

The normalized score is

$$f^*(x) = -0.2 + 0.2x_1 + 0.4x_2 - 0.4x_3 + 0.8x_4.$$

Multiplying the entire score by a positive constant preserves its decision boundary and predicted classes. The smallest normalized functional margin is 1, and the geometric margin remains $1/\|\beta^*\| = 1$.

Question 3: Peer review of a Homework 7 skill

The instructor's GitHub account, **rqzhu-aide**, will open a **GitHub Issue** in the repository containing your Homework 7 submission. It will link to **one** assigned student's repository.

1. **Use the skill.** Find the complete skill definition in the assigned student's Homework 7 report. Copy it without changes into a local `SKILL.md` file in your own folder. You do not need to install it. Give your agent the path to the file and explicitly ask it to follow that skill while analyzing the `SAheart` data, using the response and predictor coding from Homework 7. Provide the necessary data information, but no additional preferences about methods, figures, model selection, or report style. Let the skill direct the analysis.

   Monitor the time, your token use, and your weekly quota while the agent works. If the run is taking too long or using your allowance too quickly, you may send a **steer message** asking the agent to finish immediately, or stop the run and summarize what you observed. A steer message redirects an agent while it is working; learn how to send one in your AI tool before starting.

2. **Write a peer review.** Read the student's submitted report and what the skill produced in your run. Give a brief overall assessment, followed by **three major items the author could improve**. You may comment on any aspect of the skill or report. For each item, explain what you observed, why it matters, and what you recommend changing. If you stopped the run early, explain where and why. Write the review yourself, without AI assistance.

3. **Share your review.** Open a GitHub Issue in the assigned student's Homework 7 repository containing your review. Copy the same review into your Homework 8 submission report and include a link to the GitHub Issue.

https://github.com/jessicacudzich/stat432-fall2026/issues/6#issue-5916861569

Overall, the skill was very clear and easy to follow along. It explains comparing LDA and QDA, avoiding data leakage, and reporting the results. It also states which model was selected and its test error. However, here are three areas that can be improved.

Checking if the agent is reserving a test set. The skill required a separate test set, but the report from my run used all 462 observations and did not report test error. Since there were already instructions, the problem was how the agent followed them. Adding a final check list would help catch missing steps. More specific default settings. The original analysis used seed 707 and PCA with 5 or 7 components. My run used seed 433 and tried 1-8 components with cross validation. These can affect which model gets selected. Specifying default seeds, PCA options, and cross validation settings would make it more reproducible. Showing how results vary. The original report showed cross validation errors but it did not show their variability. If variability was included, it would help readers understand how much confidence to place in the ranking.